

Logistics Control Tower Architecture

Architecture document for the React, TypeScript, Vite logistics operations cockpit. This PDF summarizes the current MVP structure, data flow, API boundary, authorization model, quality approach, deployment shape, and production migration considerations.

APPLICATION Logistics Control Tower SPA	FRONTEND React 19, TypeScript, Vite	STATE Redux Toolkit and RTK Query	MOCK API JSON Server on port 3002
---	---	---	---

1. Executive Overview

The Logistics Control Tower is a single-page application that gives operations users one browser-based workspace for dashboard KPIs, shipment visibility, fleet status, exception handling, facility context, capacity planning, route planning, maps, and administration.

- Primary goal: shorten the detect-diagnose-act loop for logistics exceptions.
- Current backend: JSON Server mock API backed by `db.json` for local development and test.
- Target production direction: replace the mock API with an API gateway or backend-for-frontend while keeping the frontend API boundary stable.
- State model: RTK Query owns server state; Redux slice owns durable authentication state; components own local UI state.

2. System Context

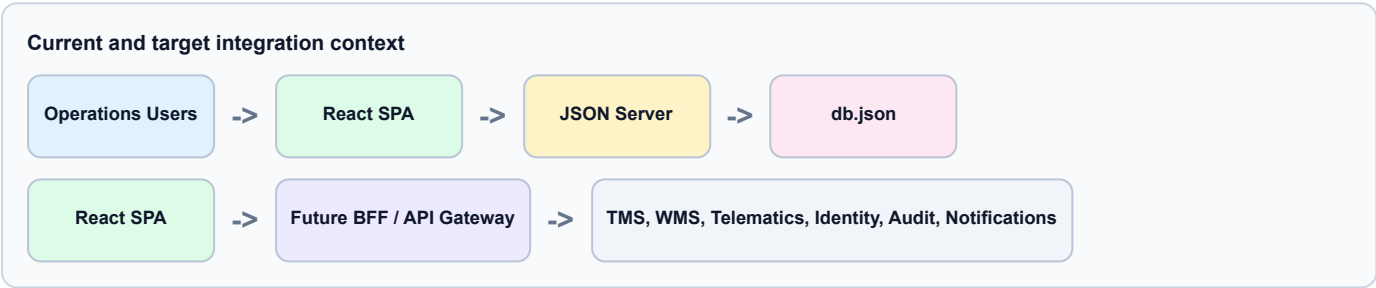
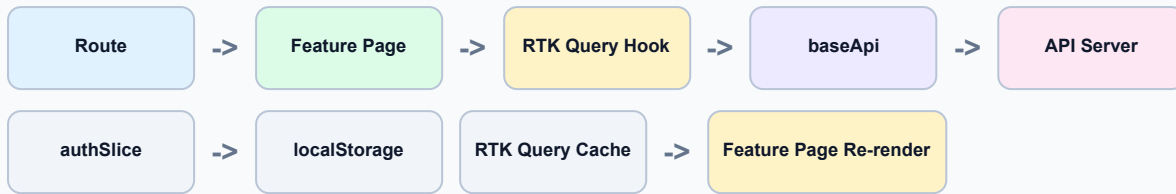


Figure 1: The MVP uses JSON Server locally; production should swap in real backend services.

3. Frontend Layers

Layer	Location	Responsibility
Bootstrap	<code>src/main.tsx, src/App.tsx</code>	Mount React, register Redux, theme, router, and i18n providers.
Routing	<code>src/app/routes.tsx, src/routes/ProtectedRoute.tsx</code>	Define the route tree, lazy-load feature pages, and enforce authentication and role access.
Layout	<code>src/layout</code>	Provide the shared app shell, navigation, and cross-page controls.
Features	<code>src/features</code>	Hold domain screens for dashboard, shipments, exceptions, fleet, facilities, capacity, routes, map, login, and admin.
Shared UI	<code>src/components</code>	Reusable controls, forms, dialogs, loaders, map components, and display primitives.
State and API	<code>src/store</code>	Redux store, auth slice, RTK Query base API, injected endpoints, and exported hooks.
Configuration	<code>src/config, src/theme, src/i18n</code>	Region settings, Material UI theme, and translations.

4. Runtime Data Flow

Browser runtime flow

- Login uses `authApi.login` to query demo users, then stores credentials through `authSlice`.
- `ProtectedRoute` checks Redux auth state and optional `allowedRoles` before rendering protected pages.
- Feature pages call generated hooks exported from `src/store`.
- RTK Query caches responses and uses tag invalidation for updates such as exception and user mutations.

5. Route Access Model

Route	Access
/dashboard, /map, /shipments, /exceptions	Any authenticated user.
/routes	Dispatcher, Operations Manager, Planner, Administrator, Read-only Viewer.
/fleet	Dispatcher, Operations Manager, Administrator, Read-only Viewer.
/facilities	Warehouse Coordinator, Operations Manager, Administrator, Read-only Viewer.
/capacity	Planner, Operations Manager, Administrator, Read-only Viewer.
/admin	Administrator only.

The route gate protects the browser experience and guides navigation. Production services must still enforce authorization server-side for every read and mutation.

6. API Boundary

`src/store/api/baseApi.ts` centralizes `fetchBaseQuery`, the API base URL, cache tag types, reducer path, and middleware integration. Domain endpoints are injected into this base API so feature components consume typed hooks without coupling to transport details.

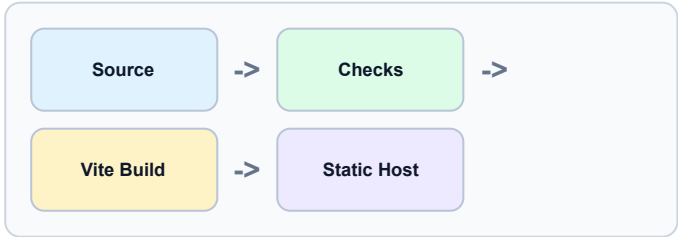
Resource	Current Use
/users	Demo login, user administration, and mock current-user lookup.
/auditEvents	Administration audit log and recorded mock actions.
/dashboardMetrics	Dashboard KPI tiles.
/shipments and /orders	Shipment grid plus client-side enrichment, search, filters, sorting, grouping, and pagination.
/vehicles	Fleet and driver status.
/exceptions	Exception queue and update mutation.
/facilities	Facility cards, events, and yard appointment context.
/capacityForecasts and /capacityScenarios	Capacity planning and what-if workflows.
/routePlans	Route planning and candidate route comparison.

7. Quality Strategy

- TypeScript build validates compile-time contracts.
- Oxlint and Prettier keep source quality and formatting consistent.
- Vitest covers utilities, slices, hooks, and feature behavior with coverage thresholds.
- Playwright covers authentication, navigation, operator journeys, screenshots, traces, and accessibility checks.
- axe-core integrates with browser tests for automated accessibility validation.

8. Deployment Shape

The frontend builds static assets into `dist/`. A static host or CDN serves those assets with SPA fallback to `index.html`. `VITE_API_BASE_URL` is embedded at build time and must point to a reachable backend API.



9. Production Readiness Notes

- Replace JSON Server authentication with a secure identity provider or backend-managed session.
- Move password validation, authorization, audit creation, and mutation validation to the backend.

- Move large-data operations such as shipment joins, search, sorting, filtering, grouping, and pagination server-side.
- Add frontend observability for runtime errors, failed API calls, latency, and critical journey health.
- Configure HTTPS, CORS, Content Security Policy, cache headers, and SPA fallback in the hosting platform.

Logistics Control Tower - Architecture

10. Architecture Decisions

Decision	Rationale
React, TypeScript, Vite	Fast local development, static deployment, strong frontend typing.
RTK Query for server state	Centralized caching, generated hooks, de-duplication, and invalidation.
JSON Server for MVP mock API	Repeatable local contracts before production integrations exist.
Protected routes for role gating	Route-level access rules remain visible and easy to audit.
Lazy-loaded feature routes	Smaller initial load and clear split points for domain modules.